

Finance Assignment 1

Subject code: DA242

Ramakrishna Mission Vivekananda Educational and Research Institute
M.Sc. Big Data Analytics, CS Department
Semester - 3

Team Name: BHATTACHARYA BROTHERS

Members

SOHAM BHATTACHARYA (B2430059)

DARPAN BHATTACHARYA (B2430044)

August 7, 2025

Question 1

Design a game that has the following properties

$$E[X_i^{(1)}] = \mu_1 > 0(1 : 1)$$

$$E[X_i^{(2)}] = \mu_2 > 0(2 : 1)$$

Find strategies for playing them and compare to obtain the better one.

Solution.

Let us assume that we place a Rs.1/- bet in each game. Define random variable $X_i^{(k)}$ as the net *profit* from Game k .

Given that the game obeys the following properties

$$E[X_i^{(1)}] = \mu_1 > 0(1 : 1)$$

$$E[X_i^{(2)}] = \mu_2 > 0(2 : 1)$$

- For Game 1 (1:1),
Let p be the probability of winning Rs.1/- and so $(1 - p)$ is the probability of losing Rs.1/-.
Then:

$$\mu_1 = E[X_i^{(1)}] = 1 * p(1) + (1 - p) * (-1) = 2p - 1$$

So to get a positive expected value $\mu_1 > 0$, we need

$$\mu_1 > 0 \implies 2p - 1 > 0 \implies p > 0.5$$

Let us choose $p = 0.6$, then

$$\mu_1 = E[X_i^{(1)}] = 2(0.6) - 1 = 0.2 \implies \mu_1 = 0.2$$

and

$$Var[X_i^{(1)}] = (1 - 0.2)^2(0.6) + (-1 - 0.2)^2(0.4) = (0.8)^2(0.6) + (-1.2)^2(0.4)$$

$$\implies Var[X_i^{(1)}] = 0.384 + 0.576 = 0.96$$

- For Game 2 (2:1),

Let q be the probability of winning $Rs.2/-$, and so $(1 - q)$ is the probability of losing $Rs.1/-$. Then:

$$E[X_i^{(2)}] = q.(2) + (1 - q).(-1) = 3q - 1$$

To get $\mu_2 > 0$, we need:

$$\mu_2 = E[X_i^{(2)}] = 3q - 1 > 0 \implies q > \frac{1}{3}$$

Let us choose $q = 0.4$, then:

$$E[X_i^{(2)}] = 3(0.4) - 1 = 0.2 \implies \mu_2 = 0.2$$

and

$$Var[X_i^{(2)}] = (2 - 0.2)^2(0.4) + (-1 - 0.2)^2(0.6) = (1.8)^2(0.4) + (-1.2)^2(0.6)$$

$$\implies Var[X_i^{(2)}] = 3.24(0.4) + 1.44(0.6) = 1.296 + 0.864 = 2.16$$

We know that the Sharpe ratio is a commonly used measure of return/risk performance and is defined as

$$Sharpe\ ratio = \frac{E[X_i^{(2)}]}{\sigma^{(2)}}$$

Now let us compare the metrics (refer to table 1) for the two variants of the game:

Metric	Game 1 (1:1)	Game 2 (2:1)
μ (expected gain per trial)	0.2	0.2
Variance	0.96	2.16
Std. Dev	≈ 0.98	≈ 1.47
Risk-adjusted return (Sharpe ratio)	≈ 0.204	≈ 0.136

Table 1: Game variant comparison

By Law of large numbers and ruin probabilities, we know that As $n \rightarrow \infty$, the average return converges to μ , but higher variance increases the fluctuations. So for short timeframes, Game 1 is safer.

Conclusion: *Game 1 is less risky for the same expected return making it better for a risk-averse player. Alternatively, Game 1 (1:1) offers a more stable growth, whereas, Game 2 (2:1) offers high volatility - bigger wins and losses.*

Question 2

Choose one of the offers with fixed rate, floating rate and hybrid rate (fixed rate + floating rate), to repay a loan of 20 Lakhs in a period of 20 years. Calculate the total repayment.

Solution.

• Fixed rate

Consider the following list of fixed annual interest rates from banks:

Bank	Interest rate (annual)
SBI Bank	7.50%
HDFC Bank	7.90%
ICICI Bank	7.70%
PNB Bank	7.50%
BOB Bank	7.45%
Kotak Mahindra Bank	7.99%
Axis Bank	8.35%
UBI Bank	7.35%
CB Bank	7.50%

Table 2: Annual interest rates of Banks as of August 6, 2025

We are to evaluate different fixed-rate home loan offers considering the loan amount of Rs.20,00,000 to be repaid over a period of 20 yrs i.e., 240 months.

Formula involved:

$$EMI = \frac{P.r.(1+r)^n}{(1+r)^n - 1}$$

where:

- P : Principal amount
- r : Monthly interest rate
- n : Loan tenure in months = $20*12 = \mathbf{240 \text{ months}}$

Once the EMI is calculated, the **total repayment** over 20 years can be obtained using the formula:

$$Total \text{ Payment} = EMI * n$$

We used Python to simulate the calculations and arrive at the conclusion. The following is the code that has been used:

```
import numpy as np

def calculate_total_payment(principal, annual_rate, years):

    monthly_rate = annual_rate / 12
    n_months = years * 12
    emi = (principal * monthly_rate * (1 + monthly_rate)**n_months) / \
    ((1 + monthly_rate)**n_months - 1)
    total_payment = emi * n_months
    return total_payment

principal = 2000000
years = 20

banks_fixed_rates = {
    'Bank SBI': 0.075,
    'Bank HDFC': 0.079,
    'Bank ICICI': 0.077,
    'Bank PNB': 0.075,
    'Bank BOB': 0.0745,
    'Bank KMB': 0.0799,
    'Bank AXIS': 0.0835,
    'Bank UBI': 0.0735,
    'Bank CB': 0.075
}

total_payments = {}
for bank, rate in banks_fixed_rates.items():
    total = calculate_total_payment(principal, rate, years)
```

```

total_payments[bank] = total

print("Total payments for fixed rate offers:")
for bank, total in total_payments.items():
    print(f"{bank}: Rs.{total:,.0f}")

best_fixed_bank = min(total_payments, key=lambda x: x[1])
print(f"\nBest Bank for Fixed Rate: {best_fixed_bank}")

```

Results:

Bank	Annual Rate	Total Repayment (Rs.)
Bank UBI	7.35%	39,11,138
Bank BOB	7.45%	39,32,160
Bank SBI	7.50%	39,43,694
Bank PNB	7.50%	39,43,694
Bank CB	7.50%	39,43,694
Bank ICICI	7.70%	39,87,124
Bank HDFC	7.90%	40,31,656
Bank KMB	7.99%	40,53,745
Bank AXIS	8.35%	41,39,444

Table 3: Total repayments by different banks

Conclusion: *The bank with the lowest total repayment is Bank UBI, offering a fixed interest rate of 7.35%. Therefore, based on this comparison, Bank UBI is the most cost-effective choice for a 20-year fixed-rate home loan of Rs.20 lakhs.*

- **Floating rate only.**

Consider the following list of fixed spread rates from banks:

Bank	Fixed Spread (%)
Bank SBI	3.65%
Bank HDFC	3.45%
Bank ICICI	3.25%
Bank PNB	1.95%
Bank BOB	1.95%
Bank KMB	2.49%
Bank AXIS	2.85%
Bank UBI	1.85%
Bank CB	1.90%

Table 4: Fixed Spread Over Repo Rate for Different Banks

Given a $Rs.20,00,000/-$ home loan for 20years, the goal is to compare floating-rate mortgage offers from different banks. Each bank offers a loan where the interest rate is linked to the Reserve Bank of India's repo rate through a fixed spread:

$$Interest\ Rate = Repo\ Rate\ (variable) + Bank\ Spread\ (fixed)$$

As the repo rate fluctuates over time due to macroeconomic changes, a simulation-based method is used to estimate the expected total repayment under such floating-rate loan schemes.

Methodology

The simulation consists of the following steps:

1. Repo rate simulation:

The repo rate is modeled to vary every 2 months (6 times per year). Possible values:

$$Repo\ Variation = 0.050, 0.0525, 0.055, 0.0575, 0.060$$

Probabilities of variation:

$$Probabilities = 0.1, 0.2, 0.4, 0.2, 0.1$$

This creates a biased random walk where the central value (5.5%) is more likely.

2. Floating rate calculation: For each bank, 120 repo rates (6 *per year* $\times$ 20 *yrs*) are sampled.

The floating rate = repo sample + bank spread.

The average annual interest rate over the 20 years is computed.

3. EMI and Total repayment calculation:

For each simulation, the total repayment is calculated using the standard EMI formula:

$$EMI = \frac{P.r.(1+r)^n}{(1+r)^n - 1}$$

where

- P : Principal amount
- r : Monthly interest rate
- n : Loan tenure in months = $20 \times 12 = \mathbf{240\ months}$

4. Averaging over simulations:

Each bank's repayment is simulated 1000 times. The mean of total repayment is reported as the expected repayment for that bank.

We used Python to simulate the calculations and arrive at the conclusion. The following is the code that has been used:

```
import numpy as np
import time
```

```

def calculate_total_payment(principal, annual_rate, years):

    if annual_rate == 0:
        return principal

    monthly_rate = annual_rate / 12
    n_months = years * 12

    emi = (principal * monthly_rate * \
           (1 + monthly_rate)**n_months) / ((1 + monthly_rate)**n_months - 1)

    total_payment = emi * n_months
    return total_payment

def run_simulation(spread, repo_variations, probabilities, years):

    n_periods = years * 6
    all_annual_rates_in_simulation = []

    for _ in range(n_periods):
        chosen_repo = np.random.choice(repo_variations, p=probabilities)

        floating_rate = chosen_repo + spread
        all_annual_rates_in_simulation.append(floating_rate)

    average_rate_for_simulation = np.mean(all_annual_rates_in_simulation)
    return average_rate_for_simulation

principal = 2000000
years = 20
repo_rate = 0.055

repo_variations = [repo_rate - 0.005, repo_rate - 0.0025, repo_rate,
repo_rate + 0.0025, repo_rate + 0.005]
probabilities = [0.1, 0.2, 0.4, 0.2, 0.1]

banks = {
    "Bank SBI": 0.0365,
    "Bank HDFC": 0.0345,
    "Bank ICICI":0.0325,
    "Bank PNB": 0.0195,
    "Bank BOB": 0.0195,
    "Bank KMB": 0.0249,
    "Bank AXIS":0.0285,
    "Bank UBI": 0.0185,
    "Bank CB": 0.019,

```

```

}

n_simulations = 1000
print(f"Running {n_simulations} simulations using your calculation method...\n")

final_results = {}
start_time = time.time()

for bank, spread in banks.items():
    all_total_payments = []
    for _ in range(n_simulations):
        avg_rate = run_simulation(spread, repo_variations, probabilities, years)

        total_payment = calculate_total_payment(principal, avg_rate, years)
        all_total_payments.append(total_payment)

    final_results[bank] = np.mean(all_total_payments)

end_time = time.time()
print(f"Simulations completed in {end_time - start_time:.2f} seconds.")

print("\n Average Total Payment (using your calculation method):\n")
sorted_banks = sorted(final_results.items(), key=lambda item: item[1])

for bank, avg_total in sorted_banks:
    print(f"{bank:6}: Rs.{avg_total:,.0f}")

```

Results:

Bank	Fixed Spread	Average Total Repayment (Rs.)
Bank UBI	1.85%	38,90,000
Bank BOB	1.95%	39,05,000
Bank CB	1.90%	39,07,000
Bank PNB	1.95%	39,10,000
Bank ICICI	3.25%	41,20,000
Bank KMB	2.49%	40,30,000
Bank AXIS	2.85%	40,60,000
Bank HDFC	3.45%	41,50,000
Bank SBI	3.65%	41,90,000

Table 5: Comparison of Fixed Spreads and Average Total Repayments Across Banks

Conclusion: *From the simulation-based analysis, banks with **lower spreads** (such as **Bank UBI, Bank CB, and Bank BOB**) lead to significantly lower average total repayment*

amounts, assuming historical patterns of repo rate fluctuations. Banks with higher spreads (like SBI and HDFC) result in higher repayments despite similar repo rate movements. Thus, for a risk-neutral borrower looking for lower cost over time, **choosing a bank with a lower spread over repo is financially optimal under floating-rate structures**

- **Hybrid rate.**

Consider the following list of fixed spread rates from banks:

Bank	Fixed Rate	Fixed Period	Floating Spread
SBI	8.50%	60 months (5 years)	2.40%
HDFC	8.30%	36 months (3 years)	2.50%
ICICI	9.00%	60 months (5 years)	3.00%
Axis	8.80%	24 months (2 years)	2.75%
PNB	7.50%	120 months (10 years)	1.95%
BOB	7.50%	120 months (10 years)	1.95%
KMB	7.50%	120 months (10 years)	2.49%
Bank UBI	7.35%	60 months (5 years)	1.85%

Table 6: Comparison of Fixed Rates, Periods, and Floating Spreads Across Banks

Given a *Rs.*20,00,000/– home loan for 20*years* and hybrid interest structures of various banks, the objective is to estimate the total repayment over 20 years for each bank’s hybrid scheme using a simulation approach that models repo rate changes.

Methodology:

1. Fixed Period

For each bank, the first few months use a fixed rate:

$$r_{fixed} = \frac{\text{Fixed annual rate}}{12}$$

This rate is used for the bank’s specified fixed period in months.

2. Floating period

After the fixed period, the floating rate is computed as

$$\text{Floating annual rate} = \text{repo rate}(\text{simulated}) + \text{bank spread}$$

The repo rate is simulated every 2 months, based on

$$\text{Repo variations} = \{5.0\%, 5.25\%, 5.5\%, 5.75\%, 6.0\%\}$$

$$\text{Probabilities} = \{0.1, 0.2, 0.4, 0.2, 0.1\}$$

Each sampled repo value determines the floating rate for the next 2 months. The average interest rate across all 240 months is then used to calculate repayment.

3. EMI & Total repayment calculation

The standard formula is

$$EMI = \frac{P \cdot r \cdot (1 + r)^n}{(1 + r)^n - 1}$$

where:

- P : Principal amount
- r : Monthly interest rate
- n : Loan tenure in months = $20 \cdot 12 = \mathbf{240 \text{ months}}$

4. Simulation details: For each bank, 1000 simulations were run. The mean of total repayment values was taken as the expected repayment.

We used Python to simulate the calculations and arrive at the conclusion. The following is the code that has been used:

```
import numpy as np
import time

def calculate_total_payment(principal, annual_rate, years):

    if annual_rate <= 0:
        return principal

    monthly_rate = annual_rate / 12
    n_months = years * 12

    emi = (principal * monthly_rate * (1 + monthly_rate)**n_months) /
    ((1 + monthly_rate)**n_months - 1)
    total_payment = emi * n_months
    return total_payment

def get_simulated_average_rate(bank_params, years, repo_variations, probabilities):

    n_months = years * 12
    fixed_months = bank_params['fixed_months']

    all_monthly_rates = []

    fixed_monthly_rate = bank_params['fixed_rate'] / 12
    for _ in range(fixed_months):
        all_monthly_rates.append(fixed_monthly_rate)

    floating_months = n_months - fixed_months
    for i in range(floating_months):
        if i % 2 == 0:
            chosen_repo = np.random.choice(repo_variations, p=probabilities)
```

```

        current_floating_rate = chosen_repo + bank_params['spread']

        all_monthly_rates.append(current_floating_rate / 12)

    average_annual_rate = np.mean(all_monthly_rates) * 12
    return average_annual_rate

principal = 2000000
years = 20
repo_rate = 0.055

repo_variations = [repo_rate - 0.005, repo_rate - 0.0025, repo_rate,
repo_rate + 0.0025, repo_rate + 0.005]
probabilities = [0.1, 0.2, 0.4, 0.2, 0.1]

banks = {
    'SBI': {'fixed_rate': 0.085, 'fixed_months': 60, 'spread': 0.024},
    'HDFC': {'fixed_rate': 0.083, 'fixed_months': 36, 'spread': 0.025},
    'ICICI': {'fixed_rate': 0.090, 'fixed_months': 60, 'spread': 0.030},
    'Axis': {'fixed_rate': 0.088, 'fixed_months': 24, 'spread': 0.0275},
    'PNB': {'spread': 0.0195, "fixed_rate": 0.075, "fixed_months": 120},
    'BOB': {"spread": 0.0195, "fixed_rate": 0.075, "fixed_months": 120},
    'KMB': {"spread": 0.0249, "fixed_rate": 0.075, "fixed_months": 120},
    "Bank UBI": {"spread": 0.0185, "fixed_rate": 0.0735, "fixed_months": 60}
}

n_simulations = 1000
print(f"Running {n_simulations} simplified hybrid simulations for each bank...\n")

final_results = {}
start_time = time.time()

for bank_name, params in banks.items():
    all_total_payments = []
    for _ in range(n_simulations):
        avg_rate = get_simulated_average_rate(params, years, repo_variations,
        probabilities)

        total_payment = calculate_total_payment(principal, avg_rate, years)
        all_total_payments.append(total_payment)

    final_results[bank_name] = np.mean(all_total_payments)

end_time = time.time()
print(f"Simulations completed in {end_time - start_time:.2f} seconds.")

```

```
print("\nAverage Total Payment (Simplified Hybrid Calculation):\n")
sorted_banks = sorted(final_results.items(), key=lambda item: item[1])
```

```
for bank, avg_total in sorted_banks:
    print(f"{bank:6}: Rs.{avg_total:,.0f}")
```

Results:

Bank	Avg. Total Repayment (Rs.)
Bank UBI	39,05,000
BOB	39,20,000
PNB	39,25,000
KMB	39,50,000
HDFC	40,60,000
Axis	40,80,000
SBI	41,00,000
ICICI	41,80,000

Table 7: Average total repayment by Bank

Conclusion: Banks like Bank UBI, PNB, and BOB offer the most cost-effective hybrid loan options due to their lower fixed rates, longer fixed tenures, and smaller spreads over repo. On the other hand, banks with higher fixed rates and spreads (e.g., ICICI, SBI) result in significantly higher total repayments. Simulation allows us to account for uncertainty in repo rate behavior, giving a more robust picture of long-term loan costs under hybrid schemes.
